ZHIYUAN MA

Master, Computer Science - UC Irvine, 2022
Bachelor, Computer Science - Shanghai Jiao Tong University, 2021

949-994-0870 zhiyuanma0520@gmail.com linkedin.com/in/zhiyuanmatech github.com/MaCoredroid

Summary: Software engineer building production GenAI systems end to end across Scale AI, AWS, and Salesforce. At Scale (Applied AI), designs enterprise finance agents with code-owned correctness: an Order-to-Cash invoice-clearing agent whose every dollar figure is a formula over cited source cells independently recomputed by a Financial Claim Verifier, gated by zero-defect hard controls and frozen-cohort evaluation; also owns the open-weight model program (serving cost estimation, ablations, cross-harness evals, customer-end deployment/throughput, post-training). At AWS Healthcare AI, designed ontology-enriched Patient Health Index and HCC suspecting pipelines over FHIR/HL7/free-text records, grounding ICD-10/HCC evidence with citations, CMS-HCC V28 mapping, and recall/cost evaluation, plus healthcare de-identification pipelines and clinical summarization agents at scale. Designed enterprise agent platforms that improved task success (82% to 94%) while cutting user-visible failures ~55% and LLM spend ~25%. Independently runs an LLM inference-systems research program (Lumo FlyWheel): first-author preprint on served tree speculative decoding for recurrent-hybrid models, custom Triton/CUDA kernels inside vLLM, a 3.04x measured decode speedup, and seven published systems write-ups.

Skills: Python, PyTorch, Transformers, NumPy/Pandas, C++, Java, Go, SQL, FastAPI, LangChain, LLM/GenAI, Clinical NLP, FHIR/HL7, ICD-10/HCC, RAG, Qdrant, Docker, Kubernetes, AWS/GCP, PostgreSQL, Temporal, BigQuery, Elastic Stack, CUDA/Triton, vLLM, Speculative Decoding, CUTLASS/FlashAttention, AWS Bedrock, Comprehend Medical

EXPERIENCE

Scale AI - Applied AI

Jul 2026 - Now

Paramount OTC Agent (Order-to-Cash invoice clearing) + Open-Weight Model Program

- Designed and authored the **Paramount OTC (Order-to-Cash) Agent**: a **read-only, advisory** GenAI agent that reconstructs disputed non-programmatic ad-sales invoices into a cited **Most Recent State (MRS)** — timeline, established-vs-reported findings, honest material gaps, and per-dollar verified claims — for a workflow where **~$180M/month** of new digital invoices fail to clear on time (**~$30M/month** non-programmatic, **2,000+** invoices uncleared beyond terms, **~50 specialists** spending **~75%** of effort on evidence-gathering across 6-7 systems, ~3 months average to clear).
- Architected **Order Selection Rules (OSR)**, the deterministic flagging layer: versioned **SQL checks over canonical structured records** that pre-filter uncleared invoices to the discrepant-looking subset and build **structured seeds** (located rows, metadata-linked candidate documents, failed lookups) — with OSR flags deliberately withheld from the agent to prevent anchoring, and a metadata-linkable-document routing path that closes the contract-pricing recall gap invisible to SQL.
- Designed the agent's **read-only tool surface** (schema / SQL query / span-returning search / fetch / calculator) with **backend-enforced constraints** — single SELECT only, no writes/DDL, capped rows — where every returned cell carries **__table__/__row_id__** handles so any value is citable, and a fixed runtime budget (bounded turns, tool calls, rows, 15-min wall clock) under which hitting a limit records a run failure, **never a manufactured conclusion**.
- Made money **machine-verifiable by construction**: every dollar figure exists only as a **numeric_claim** — a formula over cited source cells (SCALAR cell references or AGGREGATE selectors, summed from source, never by the model) — and the code-owned **Output Gate + Financial Claim Verifier** independently recomputes basis, unit, revision, and exact arithmetic against source before any specialist sees output; a figure that does not recompute **blocks the emit**.
- Added a **versioned, fail-closed LLM runtime guardrail** checking **grounding** (every material sentence entailed by its citation) and **disclosure** (no personal/non-public content carried in from source documents), which restarts the loop on any flag rather than publishing or silently redacting.
- Specified the **safety model**: zero-defect hard controls (read-only authority, one valid output per case, **100% verifier receipts** on rendered financial claims, no secret/credential echo) and an **S0-S4 severity ladder** with **zero allowance** for defects that could move cash the wrong way — motivated by a historical inspection where prompt-only verification once proposed collecting **$117,934.75** where about **$23,697.85** was supportable.
- Authored the **evaluation program**: a frozen-cohort ladder (**24 dev → 36 pilot → 60 expansion → 100 broader**), **Recall@stop** as the primary agent gate (an early stop is the one failure no downstream check catches), a deliberate code-vs-specialist measurement split in which a model-based judge is monitoring-only and **can never qualify the system to advance**, and specialist-facing MRS quality bars (useful-as-is ≥75%→≥90%, usable-after-≤5-min-fix ≥92%→≥98%, gaps handled safely 100%, citations resolve 100%).
- Designed **Phase-2 governed agentic policies** as versioned, approver-owned **data (not code)** — per-policy precision ≥95% / recall ≥80%, at most one review-only proposed action that **never grants execution authority** — scored under **anchoring-controlled reveal/suppression arms** where specialists pre-commit blind actions (blind agreement ≥44/48, wrong-way actions **0/48**).
- Defined **end-to-end release gates** — no unattended release, no single score; any hard-control breach or S4 blocks advancement — targeting a **≥50% reduction in median time from case trigger to first specialist action** via a randomized assisted-vs-manual study (20 units/arm), with wrong-way-cash incidents traced to agent output held at **zero, permanently**.
- Built on **Temporal durable workflows** (gate · repair · commit per case), **BigQuery** (SELECT-only tool layer), **Pub/Sub**, and **GCP**, with observability (AgentEx traces, Temporal history) explicitly separated from business evidence.
- Responsible for the **open-weight model program** that moves agent workloads off frontier APIs: serving **cost estimation** first, then **ablation sweeps** across open-weight models, **evaluation under different harnesses**, **customer-environment deployment and throughput** benchmarking, and **post-training (SFT/RL) to mitigate the gap** to frontier models.

Amazon - Software Engineer

Jul 2025 - Jul 2026

AWS Healthcare AI (Patient Health Index, BEACON HCC, Kariko - Beacon/Cipher, HealthScribe)

- Designed a **Patient Health Index POC**: a pre-computed, ontology-enriched clinical knowledge layer that turns multi-million-token FHIR/HL7/free-text patient records into clinician-oriented views: concept index, timeline, active problem list, medication-condition map, structured notes, and lightweight navigation tree.
- Built the index around **ICD-10, SNOMED CT, RxNorm, and LOINC** enrichment, typed clinical proposition extraction, assertion/temporality tagging, and relationship extraction (**PRESCRIBED_FOR**, **MAY_TREAT**, **MONITORED_BY**) so downstream agents load only targeted slices while preserving source citations.
- Built the **ontology enrichment pipeline**: ICD-10 → SNOMED CT crosswalk, SNOMED graph traversal (**IS_A**, **MAY_TREAT**, **FINDING_SITE**), chronic-vs-acute classification driving an **algorithmically derived active problem list** (no LLM at query time), and a medication-condition map combining SNOMED therapeutic edges with **temporal co-occurrence validation** to answer "why is the patient on this drug?" as a direct lookup.
- Split the index into a lightweight **navigation tree** (~10-20K tokens, a complete map of the record) and an on-demand **content store**, so agents answer "does this patient have X?" from the tree and load only targeted slices, cutting LLM input to a small fraction of the raw FHIR record while enforcing **per-claim citations** (DocumentReference/Condition/Observation IDs) architecturally.
- Designed a **tool-based selective-access layer** (navigate tree, load concept entry, read note, problem list, medication-condition map, timeline, concept search) with **assertion-aware proposition extraction** (confirmed / negated / hypothetical / family-history) and temporal tagging (chronic / acute / resolved) to prevent negated findings from being misread as positive, plus a **two-phase agent architecture** (fast inexpensive model for data gathering, capable model for synthesis) with pluggable multi-agent patterns.
- Created a **three-level skill-pack system** (general / agent-specific / product- and client-specific) so new products, HCC suspecting, pre-visit summaries, coding context broker, prior-auth form filling, and population queries, are **configurations of one engine rather than forks**, plus an **automated skill-generation feedback loop** (elicit requirements → draft → validate → refine) that onboards new use cases in hours to days without engineering changes.
- Designed **incremental index updates** (only new labs/encounters/notes are processed and merged in place) so per-patient enrichment cost scales with new data rather than full record size, enabling index reuse across agents, products, and population-level queries.
- Built a **BEACON HCC suspecting pipeline** over FHIR bundles and clinical notes: PatientGraph construction, Comprehend Medical seeding, Bedrock note-mining, deterministic ICD-10 merge/classification, and **CMS-HCC V28** crosswalk mapping with hierarchy suppression and citation carry-forward.
- Implemented **Stage 1 candidate mining**: deterministic PatientGraph construction (example patient: **463 FHIR resources → 601 nodes / 1,871 edges** NetworkX graph), three **Comprehend Medical** calls per note (DetectEntitiesV2 ICD-10-CM / RxNorm / SNOMED) seeding one **Bedrock note-mining** call per note that emits typed clinical propositions with assertion, temporality, **verbatim evidence spans**, and confidence, producing 60 ICD-10 candidates for the walkthrough patient.
- Enforced **deterministic safety gates**: structured FHIR Conditions trusted as-is; note-mined codes gated by assertion/temporality (dropping negated, hypothetical, family-history, and resolved findings) and a **0.70 confidence floor** waived on structured corroboration, with deterministic code-string merge unioning citations, dates, and evidence across sources.
- Built **Stage 3 agentic evidence compilation** (Strands agent) on a read-only, handwritten in-memory **FHIRIndex** with a **14-tool catalog** under an **18-call budget**, embedded CMS/MRO handbook playbooks per disease family, and an **anti-fabrication validator** that re-derives every cited excerpt against surfaced resources and drops unverifiable quotes before output.
- Built an **eval-only LLM-as-judge with an adversarial refute pass**: a four-axis rubric (mapping correctness, evidence support, hallucination, temporal validity) grounded in **literal CMS crosswalk rows** (11,927-row ICD→HCC files pasted as MATCHES / DIFFERENT so the model cannot invent mappings), at ~$0.09/HCC excluded from pipeline cost.
- Proved recall with **two independent checks**: structured-condition recall against physician-coded Conditions (gold 20 HCCs, **recall_true = 1.0**, 0 genuinely missed after hierarchy-suppression accounting) and a **free-text recall judge** over notes (0 note-only misses across 34 patients), and explained all 18 zero-HCC patients as low-acuity by re-mapping every candidate against the official CMS book.

- Characterized the **cost/latency model**: Cost(patient) = $1.52 (Stage 1 fixed) + $0.128 x #HCCs, with Stage 1 at ~84% of latency and ~91% of cost, Comprehend Medical per-account TPS as the throughput ceiling (4-account round-robin in eval), output tokens ~80% of the bill, and **prompt caching** saving ~$0.36/HCC.
- Evaluated HCC suspecting on a **34-patient** set with **39 HCCs**: Stage 1 note mining dominated latency (~84%), amortized cost was **~$1.45/HCC**, and structured-condition plus free-text recall checks found **0 genuinely missed HCCs** (**recall_true=1.0**).
- Designed a **five-layer HCC evaluation framework** with 24 metrics across ICD-10 correctness, HCC mapping, evidence faithfulness, clinical relevance/weight, MEAT breadth, corroboration, provider actionability, contradiction checks, precision, recall, F1, and fully qualified rate.
- Designed and owned a **four-layer PHI/PII de-identification pipeline** serving as shared infrastructure across the healthcare AI org: **rule/pattern matching** on clinic-specific schemas, a **token-level NER model** across 20+ PHI categories, **contextual heuristic post-processing** enforcing Safe Harbor rules (age bucketing, zip truncation, implicit-PHI detection, temporal-gap preservation, K-anonymity checks), and **LLM confidence scoring** with three-tier routing (auto-redact / human-review / human-redo).
- Calibrated confidence thresholds empirically by sweeping against false-positive curves; achieved ~5% human-review rate, saving ~95% of manual de-identification labor while prioritizing recall over precision (over-redaction preferred over under-redaction).
- Built evaluation and red-teaming workflows: precision/recall on labeled original-vs-redacted pairs, adversarial re-identification exercises with clinicians and privacy officers (given de-identified output + public data), with results feeding regression test suites.
- Owned the **end-to-end pre-visit summary generation agent** (Beacon): patient books visit → fills questionnaire → system combines questionnaire with HealthLake data (prior visits, labs, history) → generates structured clinical summary overnight via a **5-step completion pipeline** (atom extraction → hallucination judge → evidence linking → quality check → template beautification).
- Engineered **hallucination mitigation** using a separate model family to score each extracted medical fact ("atom") against source data, dropping atoms below confidence threshold ("rather hide a fact than fabricate one"); linked surviving atoms to source documents for clinician click-through verification.
- Scaled the generation pipeline to **500 concurrent jobs** across **US West and US East** regions with 5-10 min async latency; implemented retriable vs. non-retriable failure handling with graceful degradation for insufficient patient data or bad customer config.
- Solved **context window constraints** (200K token limit) with a 5-year look-back limit, recency-prioritized history, and older-document summarization into 1-2 line condensed atoms; drove migration path to 1M-context models.
- Led and won a **foundation model vs. fine-tuning architectural debate** with data: demonstrated that a fine-tuned Amazon Nova 2 Pro (trained on 10K labeled samples from one clinic) achieved only ~50% atom recall on a different clinic's data formats, while a prototype using Anthropic foundation model + prompt engineering outperformed on out-of-field generation, establishing the org's model strategy.
- Built a **clinician-feedback data flywheel**: clinician edits to AI-generated summaries captured as original+edited diffs, classified into 4 categories (incorrect facts, added facts, removals, new atoms), with high-signal cases flagged for manual review feeding few-shot prompt updates and golden regression sets.
- Operated the flywheel at **~100 samples/day** (designed for thousands/day) with a **~15% clinician atom-edit rate**, using Python-generated per-sample edit-distribution summaries so only high-signal cases (incorrect/added facts) reach human review; stored all cleaned de-identified data in **AWS HealthLake** for privacy-officer inspection and red-team access.
- Designed the **note de-identification feedback pipeline** for HealthScribe: integrated with OneMedical to collect generated-note vs. provider-edited-note triplets via S3-based ingestion, compute evaluation metrics (trigram overlap, edit distance, factual completeness), and persist de-identified artifacts in AWS Glue Iceberg tables for PM/scientist dashboarding.
- Architected **Kariko Cipher medical coding service** (ICD-10/CPT code generation): API Gateway + Lambda front-end for request validation and auth, ECS Fargate model service for Bedrock fine-tuned model invocation, with structured error handling and confidence-scored outputs.
- Delivered the **11/14 launch** by adding **CI integration tests (Hydra)** as a release gate and owning the design → deploy → test-gate path across **DynamoDB/Kinesis/Lambda/Step Functions/Glue**.
- Drove **AppSec readiness** to sign-off by coordinating with security engineers and implementing required remediations as part of the critical launch path.

- Unblocked demo/beta timelines under shifting science outputs and **LLM platform constraints** (regional availability for fine-tune/model copy) by shipping pragmatic fallbacks and extending model-service pipelines; stabilized shared-service delivery and documented safer **DynamoDB schema-change mechanisms**.

Salesforce - Software Engineer

May 2023 - Jul 2025

Agentforce (GenAI Agent Platform) - Insurance Domain Agents, Financial Services Cloud

- Built **insurance domain agents** on Agentforce as a first-wave developer (~200 agents on platform), serving insurance managers with sales summaries, customer insights, and document generation (e.g., proof-of-insurance) through a chat interface embedded in existing Salesforce workflow pages.
- Delivered **plan-then-execute agents** (**Java/Python**, **LangChain**, **Gemini**) that query Salesforce data and trigger workflows, cutting insight-surfacing time **~90%**.
- Designed a purpose-built **Apex API layer** for agent consumption that enforced **access control at the API layer** (reading caller identity and group against Salesforce org permissions rather than trusting the model), ran **all numerical aggregation server-side** (eliminating model miscalculation of totals, currency, and locale), and returned **pre-formatted summary strings** so the model's job was assembly, not generation.
- Proposed and shipped **invocation-location scoping** as a platform-wide feature: constraining the agent candidate pool from ~200 to 4-5 based on UI page context, effectively using invocation context as a Bayesian prior to reduce the classification search space.
- Worked with the platform team to add **chain-of-thought tracing on the agent router**, enabling agent developers to see why the model picked a given agent and optimize descriptions accordingly; this became a platform-wide observability capability.
- Executed **tool consolidation** after discovering the model called separate tools out of order: merged multi-step tools into single tools with parameter flags controlling behavior, eliminating sequencing failures structurally rather than via fragile prompt fixes.
- Dynamically filtered **visible tools per user group** (e.g., **get_sales_summary** manager-only), reducing the LLM's decision surface and preventing unauthorized data access.
- Instrumented **fine-grained latency/success telemetry** and ran **A/B prompt experiments**, lifting task success **82% to 94%**; added **PII masking/rehydration** into the pipeline.
- Hardened the **agent runtime** and **cost governance** (**timeouts**, **retries/backoff**, **circuit breakers**, **per-tool rate limits**, **embedding/prompt caches**, **dynamic model routing**) to cut user-visible failures **~55%** (**3.6% to 1.6%**), improve **p95 latency ~22%**, and reduce **tokens/session ~30%** and **LLM spend ~25%** with **<=1 pp** acceptance impact.
- Proposed and built an **automated evaluation framework**: golden test sets curated by humans/PMs/devs, ground-truth checking on numbers (exact match), keyword matching for required terms, cross-entropy scoring for answer quality, running as automated regression gating every prompt/tool change before deploy.
- Discovered via telemetry dashboards that conversations were too long for insurance manager workflows; responded by **expanding Apex summary templates** with additional SOQL queries and richer pre-formatted data, reducing follow-up turns.
- Worked with the platform team to build **tool-level versioning** (Agentforce only had prompt-template versioning) and implemented **shadow A/B testing in production** by forking conversations with different tool versions side by side; early adopters consistently favored the richer summary, and both became platform-wide capabilities.
- Separated the Apex layer into an **extensibility architecture**: distinct classes for API field interface, summary template, user group access control, and per-field business logic, enabling Salesforce developers at customer orgs to customize insurance agents without touching the core pipeline.

SELECTED PROJECTS

Lumo FlyWheel - LLM Inference Systems & Agentic-Benchmark Research (solo, 1x NVIDIA DGX Spark GB10) - macoredroid.github.io/Lumo_FlyWheel

Apr - Jul 2026

- Ran a **three-month single-author research program** with two pillars, LLM serving-performance engineering (vLLM internals, speculative decoding, custom GPU kernels) and authoring the **CNB-55** agentic-coding benchmark, entirely offline on one **NVIDIA GB10 Grace Blackwell** (128 GB unified memory, ~273 GB/s); shipped **~41.7K lines across 37 serving modules, ~491 scripts, 95 test modules**, and produced **one first-author preprint plus seven published systems volumes**.
- **First-authored a systems preprint**, *"GDN Tree-Scan: Served Tree Verification for Recurrent-Hybrid Language Models"* (arXiv/MLSys-style), designing the first public **served vLLM tree-speculative-decoding verifier for a Gated-DeltaNet hybrid** (Qwen3.6-27B FP8, 48 recurrent + 16 attention layers): a verifier contract combining attention ancestry, **path-local recurrent-state lineage**, SpecInfer-style residual commitment, and accepted-chain-only durable state publication, measuring **+27.0% token-weighted decode throughput in the paper's clean B=1 scope** while staying within the model's own recurrent-oracle equivalence floor.
- Published **seven illustrated systems write-ups**, each with an explicit failure museum of refuted hypotheses: Vol. I (Round-4b), Vol. II (Round-F), Vol. III (GDN Tree-Scan), Vol. IV (prefix caching), Vol. V (stateless tree), Vol. VI (one ULP), Vol. VII (keep-or-replay).
- Delivered a **3.04x measured decode speedup** (5.59 → 17.02 tok/s median; per-task up to **4.84x**) via speculative decoding (SuffixDecoding) with harness-coupled drafters on Qwen 3.5, raising cumulative token acceptance **33.5% → 78.9%**; re-ran a **132-cell ablation** on Qwen 3.6 (config A at 22.46 tok/s, +23%) and **refuted my own prior config recommendation** when its key effect failed to replicate, and rejected an FP8 KV-cache change after measuring an 8-10% decode regression.
- Wrote the GPU kernel stack: a forked **FlashAttention-2 tree-bias decode kernel**, a **branch-local Triton GDN tree-verify scan + accepted-chain replay kernel** (one recurrent state per tree node; **254 registers / 0 spill**; 36 → 6 row-touches/layer = **0.86x native HBM traffic**), and a **device-side multidraft committer** proven **byte-identical to the host reference and native at B=4 while ~22% faster**, all CUDA-graph-capturable and batch-invariant by construction, behind flags so the default serving path stays byte-for-byte unchanged.
- **Derived and validated a tree-ancestry WY recurrence** (T = inv(I+A)) for the gated-delta rule, exact at the float64 floor, and implemented it as a Triton kernel matching the dense tree solve to fp32 roundoff while running **~1.87x faster** at the microkernel level.
- Designed a **bit-exact prefix-cache state-restore contract for a recurrent hybrid** (Vol. IV), cutting **TTFT ~4x (11.8 → 2.6 s at 85-89% cache-hit)** on live agentic SWE serving, root-causing a stale snapshot row, a **kernel-realization mismatch** between chunked-prefill and recurrent-decode kernels, and an allocator that **never zeroes recycled recurrent-state pool rows**, an order-keyed corruption invisible to every seed-swept probe, caught via a previously unread logprob capture (spread 1.31 nats → 0.0 after fix).
- **Root-caused a months-old output-corruption bug** (Vol. V) that had defeated every numerical gate: committed-token attention KV was read from a **sibling branch's slot**; fixed with a re-linearization pass plus a **stateless-tree redesign** (the tree holds no persistent state, so native's stock cache owns correctness by construction), taking **garble ~40% → 0%** and restoring SWE-Bench-Verified resolve to exact native parity (8/16 = 8/16).
- Localized a speculative-tree **superset violation** (Vol. VI), a wider tree accepting fewer tokens than the tree it strictly contains, to **a single bf16 unit-in-the-last-place** of floating-point **reduction re-association** in the forked FlashAttention-2 kernel, after peeling off a 3.4x garble artifact and a ±0.25 temperature-noise band; fixed with a lossless contiguous-spine reorder proven bit-exact, restoring the invariant at the predicted margin (+0.166 vs +0.17).
- **Cut the speculative-decoding committer ~6x (~100 ms → ~17 ms)** (Vol. VII) by proving a 43 ms scratch-zeroing redundant (byte-identical with it off), replacing 48 per-layer host round-trips with batched gathers, and CUDA-graph-capturing the scan loop (>5x dispatch cut), then **corrected my own instrument**, isolating the true recurrent replay at 16.2 ms and re-attributing ~38 ms to a previously hidden sampling decision.
- Built the **first lossless Gated-DeltaNet tree suffix decode past depth-5** (accept 4.29/event, zero garble over 48 task-runs) by fusing a Snowflake Arctic suffix-decoding drafter with the model's MTP spine, measured the entire tree-vs-native step **to the millisecond** (the "keep vs replay" recurrent-state asymmetry: ~78% of the gap), and **published the honest verdict** that with a depth-matched linear MTP-11 control, **native linear MTP-5 remains the deployment throughput floor** (42.7 vs 32.9 tok/s at B=4, tied 10/16 SWE-Verified resolve).

- Stood up the full serving integration: self-hosted **vLLM 0.19.x on ARM64** with **~14 flag-gated source patches (~5.7K lines)** plus a ~7.7K-line flag-gated monkeypatch, a **Responses-API-normalizing inference proxy** so a frontier coding agent (Codex CLI) can drive the patched local model, an **SSH x86 eval-offload** recovering the ~20-35% aarch64 SWE-Bench carve-out, and a from-scratch telemetry stack (server-side step traces, per-event accept/cost traces, **GPU span timers**, pinned telemetry definitions for honest measurement under concurrency).
- Built an **agentic auto-research loop** (propose → measure → iterate → freeze) over a layered action space (**L0 kernels → L1 vLLM config → L2 request shaping → L3 LoRA**) with Optuna TPE, `@triton.autotune`, and CUTLASS inner optimizers gated by a parity-correctness substrate; profiled decode as **memory-bandwidth-bound** (FP8 GEMM = 86% of self-time, 73% of 273 GB/s) and modeled a ~5.4x concurrency ceiling.
- Ran a **SWE-Bench Verified campaign on ARM64** (Qwen 3.6-27B): tier-0 20/20 complete with 10 resolved (50%), **arm64-vs-x86 eval parity validated 17/17 with 0 flips**, a failure-mode taxonomy, and fixed-subset B=4 config comparisons with pre-registered harness fixes.
- Designed and authored **CNB-55**, an **11-track / 55-family / 275-variant** agentic-coding benchmark (30/55 families frozen) with **3-layer deterministic graders** (visible pytest <=30%, hidden behavioral/differential >=50%, trusted final-state gates), V1-V5 difficulty progression, per-track 100-pt scorecards, a **probe → harden → re-probe** calibration loop targeting a [15,25] frontier-model band, freeze + human-verification gates, anti-shortcut red-team traps, and contamination controls (pinned image digests, offline corpora).
- Caught a real **benchmark-validity failure across 208 runs**: exit-code-based grading masked a model that wrote **0 files**; root-caused to a chat-template/tool-call-parser defect and proven with a frontier control on the identical harness; built **family-clustered bootstrap CIs** and dual P_benchmark/M_training emission for the RL/SFT training flywheel.
- Practiced **publish-the-correction rigor**: retracted my own multi-spine "+6.85% lossless" win as non-distribution-preserving, corrected an acceptance headline inflated **40-124% by repeat-loop garble** from two correctness flags silently defaulting off in the production launcher (fixed structurally with a single source-of-truth flag file + fail-loud boot assertion), and documented every refuted route in per-volume failure museums.

AlphaPolicy - Probabilistic Alpha Model + Learned Portfolio Policy (PyTorch) - ai.stock.lumorik.com

2025

- Built a **two-stage multi-asset pipeline** over a ~518-ticker S&P 500-style universe plus factor assets (SPY, QQQ, VIX, BTC): **Stage A (AlphaModel)** produces daily per-ticker **discrete probability distributions** over next-day log-return bins; **Stage B (PolicyModel)** maps cached alpha summaries into daily portfolio weights via a **cross-sectional TransformerEncoder**.
- Designed the **AlphaModel architecture** as a composite encoder: **frozen Kronos-mini** sequence encoder fused with a **trainable numeric MLP** branch via a **learned token-wise sigmoid gate** (bias-initialized at -2.0 to prefer the pretrained path), feeding a causal **TimeLLM temporal backbone** (12 layers, 4 heads) with **cross-ticker attention** for cross-sectional mixing per time step.
- Trained with a **distributional objective** (cross-entropy + discrete CRPS with **tail-weighting**: `w = (1 + 10|r|)(1 + 2*1(|r|>0.03))`) to emphasize rare high-impact moves over common small-move regimes; optional differentiable Sharpe term bridges forecasting to portfolio utility.
- Implemented the **PolicyModel** as a cross-sectional Transformer (per-ticker features: `mu`, `mu_rank`, `ret_1d/5d/20d`, `held_prev`, `live`; global features: `mu` distribution stats + market return) outputting **constrained long-only weights** via temperature-scaled softmax with optional top-k cap, per-name max weight, and a **learned scalar risk gate** for regime-based exposure control.
- Optimized a **differentiable episode objective**: annualized excess Sharpe vs. SPY with embedded turnover costs (5 bps), **effective-K breadth penalty**, **alpha alignment bonus**, and **drawdown penalty** above a configurable target, all computed via deterministic end-to-end unroll.
- Designed **reproducible, contamination-resistant evaluation** with explicit **leakage controls**: train-only normalization (discovered and fixed two leakage vectors, normalization leak and feature-exposure leak in the cross-ticker dataset), **split-aware feature masking** (not just label masking), time-disjoint chronological splits (train <= 2024-06-30 / val / test), and causal attention enforcement across all model components.
- Evaluated on a **pre-2019 golden OOS set** (2010-2018, chronologically non-overlapping with training) and a **post-2025 forward eval** (2025-01 to 2025-12-15); policy eval (Run133 epoch 7): Sharpe ~1.50, max DD ~17.8%, mean turnover ~0.18, effK ~3.4.
- Built a **refreshable inference pipeline** (fetch Yahoo prices → build sequences → dump alpha signals → evaluate policy checkpoints) with **audit-friendly experiment logs** (versioned configs + run artifacts) and pinned universe manifests to prevent silent cross-ticker attention drift.
- Productized outputs into a **research workflow** that helps users scan faster, compare narratives against model-driven signals, and stay organized around daily alpha summaries.

Lumorik - AI Tutor Platform (LLM/Agents) - ai.tutor.lumorik.com 2025

- Shipped a **goal-to-plan engine** that turns user goals into structured curricula (steps + subtopics) via **SSE-streamed LLM generation** (`o4-mini`), with live progress events, subtopic-level progress tracking, and automatic study session seeding per subtopic; plan refinement re-generates steps and regenerates sessions for new subtopics.
- Built **streaming textbook generation** over SSE with timeout recovery (auto-reset if stalled >60s), **cost-limit enforcement** per user, and structured chapter blueprint prompts producing long-form instructional content with L0 chat history context injection.
- Launched **Chat with your PDFs** (BYOT) with a full **RAG pipeline**: PDF upload → chunking → embeddings → **Qdrant vector DB** retrieval → **page-prioritized, page-linked citations**; annotation-driven Q&A endpoints enable contextual responses while documents load progressively; conversations persist per-document.
- Implemented **real-time voice sessions** using **OpenAI Realtime API** (WebSocket + VAD for speech boundary detection): partial transcript streaming to client, audio persistence in **MinIO** (S3-compatible), and post-session **GPT-4 insight generation** (key concepts, action items, questions, summary) delivered over WebSocket.
- Built **voice-based study plan intake** via OpenAI Realtime WebSockets for both new plan creation and existing plan enhancement, with Langfuse tracing and structured session state management.
- Added **textbook annotations** anchored by **DOM/XPath selectors** with threaded Q&A: AI responses generated using full textbook content + structured student context profile, with model-tracked response attribution.
- Implemented **resumable study sessions** with **L0/L2 memory**: end-of-session triggers generate L0 recap, update subtopic coverage, and write L2 summaries to the study plan for cross-session continuity.
- Generated **structured memory cards** via tool-based JSON schemas for spaced-repetition workflows, stored per session.
- Built a **URL-to-Markdown ingestion pipeline** (web sessions) with background crawling, MinIO storage, source-type inference (YouTube/Twitter/Bilibili/website), and follow-on research task endpoints.
- Added **structured evaluation hooks** (grounding/citation checks and quality rubrics) to iterate on retrieval/prompting and improve answer correctness and traceability over time.
- Designed an **AI Interviewer** (Dialogue Coder Lab): a **JSON-driven agent workflow engine** with tool-calling, phase orchestration, and template overrides powering a realtime interview runtime, WebSocket + OpenAI Realtime VAD → transcript → agent tool calls → streaming TTS response, with **bidirectional code workspace sync**, filterable coding + system-design question banks, **VAD-driven barge-in handling**, and **session persistence** via workflow execution snapshots for seamless pause/resume.
- Architected the full backend on **async FastAPI** with PostgreSQL + SQLAlchemy, integrated **Langfuse tracing** for token usage/latency/cost visibility across all LLM calls, and **Keycloak** for auth.

LumoOmni - Omni-Modal Foundation Model (Vision + Audio + Text) 2026

- Designing a **staged research build** toward an omni-modal model starting from a **Qwen3-8B-Base** text-only LLM backbone, with two parallel tracks: a **continuous-embedding VLM** (Track A) for understanding/chat, and a **discrete-token omni model** (Track B) for generation.
- Built the **VLM connector** (Track A): a **Perceiver-style resampler** (64 latents, depth 2, 8 heads) compressing 729 SigLIP2 vision patches into fixed-count visual tokens, plus an **MLP projector** (512 → 16K → 4096) mapping into LLM hidden space, for **82M trainable parameters** with frozen vision tower and LLM.
- Implemented **ablation-first evaluation**: teacher-forced loss under correct/shuffled/zeroed/noise visual conditions as a promotion gate (`loss(correct) < loss(shuffled) < loss(zero/noise)`), catching injection bugs, masking errors, and "vision ignored" collapse.
- Designed the **Unified Token Interface (UTI)** (Track B): a stable ABI for converting text/image/audio to and from global token IDs with determinism, strict token-range checks, decode sanity, and **metric-based regression gates** (PSNR/SSIM for images, log-mel + SNR for audio) as first-class promotion gates.
- Implemented **trainable-rows-only** vocabulary warm-start: resizing the LLM embedding + LM head to include modality token ranges, training only the new rows, and packaging adapters as lightweight deliverables (`trainable_rows.pt` + `token_space.json` with hash verification).
- Built dataset-scale **verification pipelines**: token space hash matching, split integrity (no overlap), decode spot checks, retokenize consistency, and dataloader smoke tests before large-scale training.

LawPorter - Founding Engineer - lawporter.com Sep 2019 - Jun 2021

- Led a **5-engineer** team to architect and ship a **Kubernetes-based bankruptcy SaaS platform** serving **1K+ companies**, **50K+ claims**, and **22K creditors** with **99.99% uptime** across the full bankruptcy lifecycle, from case filing and creditor management to claims processing and distribution.
- Designed the **microservices architecture** on Kubernetes with **Istio service mesh** for traffic management, **canary deployments** for zero-downtime releases, and **Drone CI/CD** pipelines for automated build/test/deploy.
- Cut support **MTTR from 5 hours to 40 minutes** by building **Elastic/Fluentd observability dashboards** for centralized log aggregation, search, and real-time alerting across all services.
- Owned the core **claims management engine**: ingestion of creditor claim submissions, automated validation against case data, deduplication, and status tracking through the bankruptcy court workflow.
- Built secure **multi-tenant data isolation** to handle sensitive financial and legal data across hundreds of concurrent bankruptcy cases with strict access controls.